My Project

Generated by Doxygen 1.16.1

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 3

Class Documentation

3.1 Population Class Reference

Public Member Functions

- Population (size_t pop_size, size_t dimensions)

Public Attributes

- vector< vector< float > > **population**
- vector< float > **fitness**

3.1.1 Constructor & Destructor Documentation

3.1.1.1 Population()

```
Population::Population (
            size_t pop_size,
            size_t dimensions)
```

Constructor for Population initializes population matrix to be of size pop_size x dimensions and fitness vector to size pop_size

Parameters

in	*pop_size,dimensions*	

The documentation for this class was generated from the following files:

- Population.h
- Population.cpp

Chapter 4

File Documentation

4.1 DifferentialEvolution.h

```
00001 // Name: Harrison Ingram-Bate
00002 #ifndef DIFFERENTIAL_EVOLUTION
00003 #define DIFFERENTIAL_EVOLUTION
00004
00005 #include <memory>
00006 #include <random>
00007 #include <chrono>
00008 #include "Population.h"
00009
00010 typedef std::unique_ptr<std::vector<std::uniform_real_distribution<float>>> Distributions;
00011
00017 Population DifferentialEvolution(Distributions distribution_vec,
00018                                  float (*fitness)(const vector<float>&),
00019                                  size_t result_pop_size,
00020                                  size_t gen_pop_size,
00021                                  float crossover,
00022                                  float mutation,
00023                                  float lambda,
00024                                  unsigned int generations,
00025                                  unsigned int strategy
00026                                  );
00027
00033 vector<float> DEbest1exp(Population& current_gen,
00034                          uniform_int_distribution<int>& pop_index_distribution,
00035                          uniform_int_distribution<int>& dimension_index_distribution,
00036                          uniform_real_distribution<float>& crossover_distribution,
00037                          mt19937& rand_gen,
00038                          int curr_index,
00039                          int best_index,
00040                          float crossover,
00041                          float mutation
00042                          );
00043
00049 vector<float> DErand1exp(Population& current_gen,
00050                          uniform_int_distribution<int>& pop_index_distribution,
00051                          uniform_int_distribution<int>& dimension_index_distribution,
00052                          uniform_real_distribution<float>& crossover_distribution,
00053                          mt19937& rand_gen,
00054                          int curr_index,
00055                          float crossover,
00056                          float mutation
00057                          );
00058
00064 vector<float> DErandbest1exp(Population& current_gen,
00065                          uniform_int_distribution<int>& pop_index_distribution,
00066                          uniform_int_distribution<int>& dimension_index_distribution,
00067                          uniform_real_distribution<float>& crossover_distribution,
00068                          mt19937& rand_gen,
00069                          int curr_index,
00070                          int best_index,
00071                          float crossover,
00072                          float mutation,
00073                          float lambda
00074                          );
00075
00081 vector<float> DEbest2exp(Population& current_gen,
00082                          uniform_int_distribution<int>& pop_index_distribution,
```

```
00083                           uniform_int_distribution<int>& dimension_index_distribution,
00084                           uniform_real_distribution<float>& crossover_distribution,
00085                           mt19937& rand_gen,
00086                           int curr_index,
00087                           int best_index,
00088                           float crossover,
00089                           float mutation
00090                           );
00091
00097 vector<float> DErand2exp(Population& current_gen,
00098                           uniform_int_distribution<int>& pop_index_distribution,
00099                           uniform_int_distribution<int>& dimension_index_distribution,
00100                           uniform_real_distribution<float>& crossover_distribution,
00101                           mt19937& rand_gen,
00102                           int curr_index,
00103                           float crossover,
00104                           float mutation
00105                           );
00106
00112 vector<float> DEbest1bin(Population& current_gen,
00113                           uniform_int_distribution<int>& pop_index_distribution,
00114                           uniform_int_distribution<int>& dimension_index_distribution,
00115                           uniform_real_distribution<float>& crossover_distribution,
00116                           mt19937& rand_gen,
00117                           int curr_index,
00118                           int best_index,
00119                           float crossover,
00120                           float mutation
00121                           );
00122
00128 vector<float> DErand1bin(Population& current_gen,
00129                           uniform_int_distribution<int>& pop_index_distribution,
00130                           uniform_int_distribution<int>& dimension_index_distribution,
00131                           uniform_real_distribution<float>& crossover_distribution,
00132                           mt19937& rand_gen,
00133                           int curr_index,
00134                           float crossover,
00135                           float mutation
00136                           );
00137
00143 vector<float> DErandbest1bin(Population& current_gen,
00144                           uniform_int_distribution<int>& pop_index_distribution,
00145                           uniform_int_distribution<int>& dimension_index_distribution,
00146                           uniform_real_distribution<float>& crossover_distribution,
00147                           mt19937& rand_gen,
00148                           int curr_index,
00149                           int best_index,
00150                           float crossover,
00151                           float mutation,
00152                           float lambda
00153                           );
00154
00160 vector<float> DEbest2bin(Population& current_gen,
00161                           uniform_int_distribution<int>& pop_index_distribution,
00162                           uniform_int_distribution<int>& dimension_index_distribution,
00163                           uniform_real_distribution<float>& crossover_distribution,
00164                           mt19937& rand_gen,
00165                           int curr_index,
00166                           int best_index,
00167                           float crossover,
00168                           float mutation
00169                           );
00170
00176 vector<float> DErand2bin(Population& current_gen,
00177                           uniform_int_distribution<int>& pop_index_distribution,
00178                           uniform_int_distribution<int>& dimension_index_distribution,
00179                           uniform_real_distribution<float>& crossover_distribution,
00180                           mt19937& rand_gen,
00181                           int curr_index,
00182                           float crossover,
00183                           float mutation
00184                           );
00185
00186 #endif
```

4.2 ParticleSwarm.h

```
00001 #ifndef PARTICLE_SWARM
00002 #define PARTICLE_SWARM
00003
00004 #include <memory>
00005 #include <random>
00006 #include <chrono>
```

```
00007 #include "Population.h"
00008
00009 typedef std::unique_ptr<std::vector<std::uniform_real_distribution<float»> Distributions;
00010
00011 Population RepeatedParticleSwarm(Distributions distribution_vec,
00012                           float (*fitness)(const vector<float>&),
00013                           size_t result_size,
00014                           size_t particles,
00015                           float c1,
00016                           float c2,
00017                           float slowing_factor,
00018                           int generations);
00019
00020 Distributions ParticleSwarm(Distributions distribution_vec,
00021                             float (*fitness)(const vector<float>&),
00022                             uniform_real_distribution<float>& zero_to_one,
00023                             mt19937& rand_gen,
00024                             Population& particle_gen,
00025                             Population& pBest,
00026                             vector<vector<float» velocity,
00027                             int& global_best,
00028                             float c1,
00029                             float c2,
00030                             float slowing_factor,
00031                             int iterations);
00032
00033 #endif
```

4.3 Population.h

```
00001 // Name: Harrison Ingram-Bate
00002 #ifndef POPULATION
00003 #define POPULATION
00004 #include <vector>
00005 #include <random>
00006 using namespace std;
00007
00008 class Population {
00009 public:
00015     Population(size_t pop_size, size_t dimensions);
00016
00017     vector<vector<float» population;
00018     vector<float> fitness;
00019
00020 };
00021
00022 #endif
```

4.4 Problem.h

```
00001 // Name: Harrison Ingram-Bate
00002 #ifndef PROBLEM
00003 #define PROBLEM
00004 #include <vector>
00005 using namespace std;
00006
00012 float Schwefel(const vector<float>& vec);
00013
00019 float FirstDeJong(const vector<float>& vec);
00020
00026 float Rosenbrock(const vector<float>& vec);
00027
00033 float Rastrigin(const vector<float>& vec);
00034
00040 float Griewangk(const vector<float>& vec);
00041
00047 float SineEnvelope(const vector<float>& vec);
00048
00054 float StretchedV(const vector<float>& vec);
00055
00061 float AckleyOne(const vector<float>& vec);
00062
00068 float AckleyTwo(const vector<float>& vec);
00069
00075 float EggHolder(const vector<float>& vec);
00076 #endif
```

Index